Guía para entender y defender Bicipedia

Grupo 3 · Laboratorio 1 · Proyecto de Ingeniería de Software II

Esta guía sirve para que podamos entender el proyecto, practicarlo y explicar su funcionamiento con nuestras propias palabras. Bicipedia es una tienda académica de servicios y accesorios de bicicleta: permite crear, buscar, modificar y eliminar pedidos mientras la pestaña permanece abierta. Al recargar, los pedidos desaparecen porque viven en memoria JavaScript.

La revisión del 10 de octubre de 2026 encontró los requisitos funcionales implementados y volvió a aprobar los 14 grupos de pruebas. No hay cifrado, cuentas, contraseñas, pagos reales ni base de datos. Quedan por realizar la comprobación en el XAMPP del equipo de defensa, el ensayo del grupo y la identificación honesta de los aportes de cada integrante.

Nuestro equipo

- Carlos Andrés Chavarría Quirós
- Carlos Josue Angúlo Xie
- Marco Vinicio Miranda Miranda
- Michael Aramis Cabrera Hodgson

La versión que debemos abrir

La aplicación revisada está en la rama `codex/laboratorio-1-bicipedia`, commit `0ceddcb`, del PR 1. Al revisar, el PR continúa en borrador y `main` contiene Semana 1. Para estudiar, usar el ZIP `GRUPO03_BICIPEDIA_E5.zip` o seleccionar la rama indicada antes de descargar desde GitHub. Abrir el proyecto antiguo impediría demostrar este laboratorio.

Cómo estudiar esta guía

Primero leer el alcance y la instalación. Después ejecutar la demostración sin mirar el código. Finalmente relacionar cada acción con su función y practicar las preguntas. No hace falta memorizar todas las líneas: debemos entender dónde están los datos, quién los modifica y cómo llegan a la pantalla.

Ruta de lectura: páginas 2 y 3 para el alcance, 4 para abrir, 5 a 11 para entender el código, 12 a 14 para ensayar y 15 para verificar la preparación.

El mensaje del docente asigna al Grupo 3 la defensa del 12 de octubre de 2026 a las 6:00 p. m. Esa información procede del mensaje recibido; revisar cualquier actualización del profesor.

Qué exige el laboratorio y dónde se cumple

El alcance definitivo es el de `Laboratorio 1.pdf`. La siguiente relación permite mostrar evidencia concreta sin prometer módulos de un sistema comercial.

Requisito	Dónde se implementa	Comprobación
Propuesta propia dentro de un dominio permitido	Tienda virtual Bicipedia	Inicio y catálogo coherentes
Cinco páginas o secciones	`index.php`	Inicio, catálogo, solicitud, solicitudes y dashboard
HTML semántico y accesible	`index.php`	Regiones, etiquetas, fieldsets y tabla semántica
CSS externo con Flexbox y Grid	`css/styles.css`	Adaptación entre 320 y 1280 px
Catálogo inicial en JavaScript	`js/datos-iniciales.js`	Cuatro productos precargados
Registros en un arreglo temporal	`estado.solicitudes` en `js/app.js`	Navegar conserva; recargar reinicia
Crear y consultar registros	`crearSolicitud` y `renderizarSolicitudes`	El pedido aparece sin recargar
Buscar y mostrar falta de coincidencias	`renderizarSolicitudes`	Búsqueda existente e inexistente
Editar el registro correcto	`cargarEdicion` y `editarSolicitud`	Cambia datos y conserva ID
Eliminar con confirmación	`eliminarSolicitud`	Cancelar conserva; aceptar elimina
Validar y recalcular	`validarPedido`, `calcularTotales` y PHP	Campos, cantidades e importes revisados
PHP mediante POST sin guardar datos	`php/validar.php`	JSON correcto y errores de validación
Reinicio y estados vacíos	Inicialización y renderizado	Carrito, lista y dashboard en cero
Documentación y evidencia	README, docs y evidencias	ZIP, capturas y pruebas disponibles

Fuentes: Laboratorio 1, páginas 2, 3 y 5 a 9. Las funciones del laboratorio están implementadas. La apertura directa bajo Apache/XAMPP queda pendiente de comprobar en el equipo de exposición; las pruebas actuales usaron el servidor de desarrollo de PHP. La defensa y los aportes personales los debe completar el grupo.

Qué está permitido y qué queda fuera

El laboratorio permite personalizar nombres, campos y estilo. Por eso Bicipedia conserva sus servicios, añade un accesorio de bicicleta y usa pedidos como operación principal de una tienda virtual. El carrito incluye cantidades, subtotal, impuesto y total, tal como propone ese dominio.

Opciones que sí aparecen en el enunciado

El filtro por categoría, el carrito vacío y el botón para vaciarlo están entre las ideas opcionales de la tienda virtual en la página 3. Son opciones permitidas, no requisitos obligatorios. Los filtros de estado y los conteos resumen los mismos registros temporales. No abren un módulo diferente del proyecto.

La modalidad de retiro o domicilio, los precios, la tasa simulada del 13 %, el recargo de CRC 2.500 y el límite de 20 unidades son decisiones de la propuesta. El profesor no fija esos valores. Se usan para demostrar formularios, reglas y cálculos; no son condiciones reales del negocio ni requisitos que debamos atribuir al docente.

Detalles técnicos de apoyo

Mostrar texto con `textContent`, rechazar productos inválidos, impedir un envío doble y limitar la espera del POST ayudan a que el flujo existente funcione. Son decisiones de implementación. No constituyen cifrado ni un sistema de seguridad adicional. Para la defensa basta entender su propósito y ubicar el código.

Funciones que no se han incorporado

No hay cifrado de datos, registro de usuarios, inicio de sesión, contraseñas, roles, pagos, facturación, inventario real ni notificaciones. Tampoco hay base de datos, archivos que guarden pedidos, almacenamiento del navegador o servicios externos. Esos módulos no se necesitan para este laboratorio.

Por qué no copiamos todo lo visto en clase

TaskFlow muestra almacenamiento local y consultas externas; el ejemplo de PHP muestra base de datos y sesiones. Esos ejemplos enseñan conceptos, pero Laboratorio 1 prohíbe su persistencia. Tomamos arreglos, eventos, DOM, validación y POST. La guía de Java explica un motor de pedidos aparte; este laboratorio especifica HTML, CSS, JavaScript, PHP y XAMPP, y no pide conectar Java ni Maven a Bicipedia.

Cómo abrir el proyecto para practicar

Preparar XAMPP

1. Descomprimir `GRUPO03_BICIPEDIA_E5.zip` en una carpeta de trabajo.
2. Crear `C:\xampp\htdocs\bicipedia` si XAMPP está instalado en `C:\xampp`.
3. Copiar el contenido de `proyecto` dentro de esa carpeta. Debe existir directamente `C:\xampp\htdocs\bicipedia\index.php`; evitar una carpeta `proyecto` adicional en medio.
4. Abrir el panel de XAMPP y presionar Start en Apache. Para este laboratorio no es necesario iniciar MySQL.
5. Abrir el navegador y escribir `http://localhost/bicipedia/index.php`.
6. Comprobar que aparezca el inicio, abrir Catálogo y agregar un producto. Crear un pedido ficticio para verificar que PHP funciona.
7. Mantener esa pestaña mientras se practica. Recargar solo al final, cuando se demuestre la pérdida de datos.

Si XAMPP está instalado en otra carpeta, usar su directorio `htdocs`. Si Apache usa el puerto 8080, la dirección será `http://localhost:8080/bicipedia/index.php`. Consultar el puerto del equipo, no cambiarlo a ciegas.

Qué no necesitamos hacer para ejecutar el sitio

No se instala Node ni Playwright para usar Bicipedia. Son herramientas opcionales de pruebas. No se importa SQL, no se abre phpMyAdmin y no se compila Java. JavaScript se ejecuta en el navegador y PHP en el servidor local.

Un servidor Python sirve archivos, pero no ejecuta PHP. Abrir `index.php` con doble clic o usar GitHub Pages tampoco ejecuta su validación. La dirección debe empezar por `http://localhost` o por una dirección de servidor PHP local.

Si algo falla

- Error 404: comprobar carpeta, nombre y URL; buscar `index.php` en la ruta exacta.
- El navegador muestra código PHP: abrir mediante Apache/PHP, no como archivo local.
- No se puede validar el pedido: comprobar que Apache siga activo y que exista `php/validar.php`.
- El pedido desapareció: revisar si se recargó, cerró o abrió otra pestaña. Es el comportamiento previsto.
- No aparece un registro: vaciar búsqueda y elegir Todos en el filtro.

Cómo se conectan las piezas

La pantalla y el servidor realizan tareas distintas. El navegador muestra el sitio y mantiene los pedidos en memoria. Apache recibe las peticiones locales y ejecuta PHP cuando se solicita la validación.

Cuatro responsabilidades

HTML coloca títulos, secciones, formularios, etiquetas, botones y tabla. **CSS** define colores, tamaños, distribución y adaptación a móvil. **JavaScript** atiende las acciones, maneja arreglos, calcula y vuelve a dibujar resultados. **PHP** recibe el POST, valida y devuelve una respuesta sin conservar los pedidos.

El recorrido de un pedido

1. Elegimos un producto en Catálogo.
2. JavaScript lo agrega al arreglo del carrito.
3. La pantalla del carrito y sus importes se actualizan.
4. Completamos el formulario y presionamos Crear solicitud.
5. JavaScript revisa los campos antes de enviar.
6. `fetch` envía los campos a `php/validar.php` mediante POST.
7. PHP revisa los datos y responde JSON.
8. Si PHP confirma, JavaScript agrega el pedido al arreglo y actualiza tabla y dashboard.

Si la validación falla, el pedido no entra al arreglo. El mensaje explica qué corregir; el carrito sigue disponible.

Las cinco secciones usan una misma página

Inicio presenta la idea; Catálogo permite elegir; Nuevo pedido reúne carrito y formulario; Solicitudes permite consultar y gestionar; Dashboard resume las solicitudes actuales. En el código, el catálogo tiene el identificador `servicios` y el formulario `solicitud`.

Al pulsar una opción cambia una ancla como `#solicitudes`. `mostrarSeccion` oculta las otras secciones con `hidden` y muestra la elegida. No vuelve a cargar el documento. Por eso el arreglo sigue disponible. Los HTML antiguos solo redirigen a esta aplicación; no conviene usarlos para navegar durante la demostración.

Dónde viven los datos

Un **arreglo** es una lista. Un **objeto** agrupa datos con nombres. Nuestro estado combina listas de productos seleccionados y pedidos con variables que indican qué operación está en curso.

Este es el objeto real al iniciar `js/app.js`:

```
const estado = {
  solicitudes: [], carrito: [], editandoId: null,
  siguienteId: 1, enviando: false
};
```

`solicitudes` es la lista de pedidos creados. `carrito` es la selección que todavía estamos preparando. `editandoId` es el código del pedido que se modifica; `null` significa que estamos creando uno nuevo. `siguienteId` genera códigos consecutivos. `enviando` indica que esperamos la respuesta PHP.

El carrito y la solicitud no son la misma lista

Un ítem del carrito tiene `productoId` y `cantidad`. El nombre y precio se consultan en el catálogo. La solicitud validada reúne ID, nombre, apellido, correo, teléfono, modalidad, fecha, dirección, comentarios, estado, ítems e importes. PHP completa nombres y precios desde su catálogo.

El catálogo está escrito en `js/datos-iniciales.js`: por eso reaparece después de recargar. Las solicitudes empiezan con `[]`: por eso vuelven a estar vacías. Abrir una nueva pestaña crea otro estado independiente; las pestañas no comparten pedidos.

Cómo se generan los códigos

El primer pedido es BIC-001, el segundo BIC-002. Editar conserva ese código. Eliminar BIC-001 no convierte BIC-002 en BIC-001. Dentro de la misma sesión, el siguiente pedido sigue el contador. Recargar vuelve a iniciar el contador en uno.

Memoria no significa sesión PHP. El servidor no guarda el arreglo. Los archivos JSON de configuración y resultados de pruebas tampoco guardan pedidos del usuario. **JSON es un formato de texto para representar datos; convertir a JSON no cifra ni almacena por sí mismo.**

Cómo se crea una solicitud

`procesarFormulario` conecta el formulario, la validación PHP y el arreglo. Se ejecuta porque el formulario tiene un evento `submit` registrado con `addEventListener`.

Qué hace antes de enviar

`preventDefault()` evita que el formulario recargue la página de la forma habitual. `leerFormulario()` recoge los campos con `FormData`. `validarPedido()` devuelve un objeto con errores: si está vacío, se puede continuar; si contiene errores, `mostrarErrores()` los coloca junto al campo y en un resumen.

El carrito se copia para trabajar con los ítems del envío actual. `JSON.stringify(items)` convierte esa lista en texto y se incorpora al POST. No es una copia guardada en disco.

Esta es la llamada real al servidor:

```
const respuesta = await fetch('php/validar.php', {
  method: 'POST', body: campos,
  signal: controlador.signal, cache: 'no-store'
});
const resultado = await respuesta.json();
```

`await` espera la respuesta sin detener todo el navegador. `respuesta.json()` interpreta el texto JSON que PHP devolvió. `signal` permite cancelar una espera excesiva y `no-store` pide no cachear esa respuesta.

Qué hace después

Si los datos son correctos, `crearSolicitud(resultado.datos)` genera el ID y agrega al inicio de la lista mediante `unshift`. Después limpia el formulario y carrito, quita filtros anteriores, ejecuta `renderizarSolicitudes()` y `renderizarDashboard()`, y abre Solicitudes.

Si está editando, llama `editarSolicitud` con el ID existente. Si el servidor falla, muestra un mensaje y conserva el carrito; no agrega una solicitud sin validar. Durante la espera bloquea el formulario para evitar dos envíos simultáneos. El tiempo máximo de espera configurado es de 15 segundos, una decisión técnica del proyecto.

Cómo buscar editar y eliminar

La búsqueda, edición y eliminación trabajan sobre `estado.solicitudes`. No consultan una base de datos. Estos nombres de funciones son útiles para localizar el código durante la defensa.

Buscar y listar

`renderizarSolicitudes()` usa `filter` para obtener los pedidos que coinciden con texto y estado. La búsqueda considera ID, nombre, apellido, correo y nombres de productos. Quita diferencias entre mayúsculas y acentos para facilitar coincidencias.

Después crea filas y reemplaza el contenido de la tabla con `replaceChildren`. La lista original conserva todos los registros: filtrar solo cambia qué filas vemos. `buscarSolicitud(id)` usa `find` para obtener un pedido concreto cuando vamos a editarlo o eliminarlo.

Editar sin cambiar el código

`cargarEdicion(id)` busca el pedido, copia sus ítems al carrito y rellena los campos. La copia evita que modificar el borrador altere inmediatamente el original. Cancelar limpia el borrador y conserva la solicitud. Guardar vuelve a validar con PHP.

La sustitución real usa `map`:

```
estado.solicitudes = estado.solicitudes.map(
  (s) => s.id === id ? { id, ...datos } : s
);
```

Para cada elemento pregunta si tiene el ID elegido. Si coincide, devuelve los nuevos datos con el mismo ID. Si no coincide, conserva ese elemento. `...datos` incorpora las propiedades del objeto validado.

Eliminar solo cuando se confirma

`eliminarSolicitud(id)` usa `window.confirm`. Cancelar hace que termine sin cambios. Aceptar construye una lista que conserva todos los pedidos excepto el ID seleccionado:

```
estado.solicitudes = estado.solicitudes.filter(
  (s) => s.id !== id
);
```

Después actualiza tabla y dashboard. `find` encuentra uno, `filter` selecciona varios y `map` transforma cada elemento. Estas tres ideas explican buena parte del CRUD, que significa crear, consultar, actualizar y eliminar.

Qué valida JavaScript y qué valida PHP

JavaScript permite corregir errores sin enviar una petición innecesaria. PHP vuelve a comprobar el POST porque una petición puede llegar con datos diferentes a los permitidos por la pantalla. En ambos casos la validación forma parte del laboratorio.

Reglas de nuestro formulario

Nombre y apellido requieren al menos dos caracteres. El correo debe tener un formato válido. La fecha debe ser hoy o posterior y existir en el calendario. Debe haber al menos un producto con cantidades enteras de 1 a 20. Se acepta una de las modalidades y uno de los estados definidos. La casilla confirma el carácter temporal del pedido.

Teléfono y comentarios son opcionales. El teléfono, si se escribe, debe tener el formato permitido. Domicilio exige una dirección de al menos ocho caracteres; retiro no exige dirección. También hay límites de longitud. Son reglas elegidas para el formulario, no números fijados por el profesor.

El trabajo del servidor

`php/validar.php` lee `$_POST`, elimina espacios extremos, comprueba tipos y campos, interpreta los ítems JSON y consulta `catalogo()` en `php/catalogo.php`. Rechaza productos inexistentes, repetidos o cantidades inválidas. Calcula desde sus precios, no desde un total escrito por el cliente.

Al terminar responde JSON mediante `responder`. Las respuestas relevantes son:

Código	Qué significa en Bicipedia
200	Datos correctos y resumen calculado
422	Datos que debemos corregir
405	Se intentó usar otro método distinto de POST
413	El cuerpo enviado supera el límite de tamaño

Después de responder, esa ejecución de PHP termina. **Quien conserva el pedido visible es JavaScript.** No se escriben registros, no se inicia una sesión PHP y no se conecta a MySQL. `textContent` muestra lo escrito por el usuario como texto; no lo cifra ni lo convierte en HTML ejecutable.

Cómo se calculan los importes y el dashboard

El catálogo de ejemplo contiene mantenimiento por CRC 12.900, frenos y transmisión por CRC 7.500, diagnóstico por CRC 3.500 y kit por CRC 9.500. Esos precios no son cotizaciones del negocio.

`calcularTotales(items, modalidad)` consulta cada producto y multiplica precio por cantidad. Suma todas las líneas, calcula el impuesto de ejemplo del 13 % y agrega la recolección ficticia solo cuando el carrito tiene ítems y se elige domicilio.

Un ejemplo que podemos explicar

Concepto	Operación	Importe CRC
Dos mantenimientos	12.900 × 2	25.800
Impuesto simulado	25.800 × 13 / 100	3.354
Recolección ficticia	Solo a domicilio	2.500
Total a domicilio	25.800 + 3.354 + 2.500	31.654
Total con retiro	25.800 + 3.354	29.154

Por dentro se usan céntimos: CRC 12.900 se representa como 1.290.000 céntimos. Se trabaja con enteros y se redondea el impuesto una vez por pedido. Al mostrar moneda se divide entre 100. PHP realiza el mismo cálculo y las pruebas comparan ambos resultados.

El dashboard resume el arreglo actual

`estado.solicitudes.length` da el total de pedidos. `filter(...).length` cuenta cada estado. `reduce` suma `totalCentimos` de todas las solicitudes. El importe incluye todos los estados; no representa pagos cobrados, ganancias ni facturación.

Después de crear, guardar una edición o confirmar una eliminación se llama `renderizarDashboard()`. Por eso sus números coinciden con la lista, en lugar de ser datos fijos escritos en HTML.

Cambiar solo la fecha no modifica el precio de un servicio: en esta tienda el costo depende del producto, cantidad y modalidad. Cambiar cantidad o modalidad sí cambia el total. Editar un estado mueve los conteos correspondientes.

Dónde encontrar cada parte del proyecto

Este mapa es para estudiar y localizar funciones si el profesor pregunta por el código. Abrir la carpeta en un editor y buscar el nombre de la función, sin memorizar números de línea.

Archivo	Qué debemos buscar
`index.php`	Cinco secciones, formulario, tabla, enlaces a CSS y JavaScript
`css/styles.css`	Flexbox, Grid, media queries, foco y tabla desplazable
`js/datos-iniciales.js`	productos, estados, MAX_CANTIDAD, calcularTotales y fechaHoy
`js/validaciones.js`	validarPedido y sus mensajes
`js/app.js`	estado, eventos, navegación, CRUD, renderizado y fetch
`php/catalogo.php`	Nombres y precios usados en servidor
`php/validar.php`	POST, comprobaciones, cálculo y JSON
`tests/laboratorio.mjs`	Pruebas automáticas opcionales
`evidencias/`	Capturas e informe de la implementación

Palabras del código en lenguaje sencillo

DOM es la representación de la página que JavaScript puede consultar y modificar. **Evento** es una acción, como click, input o submit. `addEventListener` conecta esa acción con una función. **Renderizar** significa dibujar de nuevo la parte de pantalla a partir de los datos actuales.

POST es el método usado para enviar los campos al servidor. **JSON** es el texto estructurado de la respuesta y de los ítems enviados; no es una base de datos. **Responsive** significa que la distribución se adapta al ancho. **Media query** aplica reglas CSS según condiciones como el ancho de pantalla.

El archivo termina llamando las funciones que dibujan catálogo, carrito, solicitudes y dashboard. Los botones creados dinámicamente se atienden desde sus contenedores: el evento identifica qué producto o ID se pulsó. Los atributos `data-*` transportan esos identificadores en los elementos HTML.

Los archivos dentro de `docs/evidencias/` son capturas históricas de Semana 1. Para el laboratorio usar `evidencias/` y los resultados de la revisión actual; no presentar la tabla estática antigua como si fuera el CRUD.

Demostración completa para la defensa

El enunciado pide crear dos registros, buscar y editar uno, eliminar otro y recargar. Este recorrido cubre esas acciones. Puede ensayarse en unos diez minutos; no es un tiempo impuesto por el profesor. Usar la fecha actual o una futura al practicar.

1. Abrir la aplicación limpia. Explicar la tienda ficticia, sus usuarios y las cinco secciones. Mostrar el aviso de datos temporales.
2. En Catálogo agregar un mantenimiento. En Nuevo pedido escribir Ana, Mora y `ana@example.com`; elegir retiro, fecha válida y marcar la casilla. Crear BIC-001. Total esperado CRC 14.577.
3. Volver al catálogo y agregar un kit. Crear BIC-002 para Luis Vega, `luis@example.com`, con domicilio y una dirección ficticia suficiente. Total esperado CRC 13.235. Dashboard: dos pendientes y total CRC 27.812.
4. Buscar BIC-001. Mostrar que aparece solo Ana. Buscar una palabra inexistente y explicar el mensaje vacío. Limpiar la búsqueda antes de continuar.
5. Editar BIC-001: cantidad dos, modalidad retiro y estado En atención. Guardar. El ID sigue siendo BIC-001 y su total pasa a CRC 29.154. Dashboard: dos pedidos, uno pendiente y uno en atención; total CRC 42.389.
6. En BIC-002 pulsar Eliminar y cancelar. Mostrar que todavía existe. Repetir y aceptar. Queda BIC-001 y el dashboard suma CRC 29.154.
7. Navegar entre secciones y comprobar que el pedido permanece. Mostrar `estado.solicitudes`, una función CRUD y `php/validar.php` en el editor.
8. Recargar al final. Lista y carrito vacíos, métricas en cero. Explicar que JavaScript volvió a ejecutar el estado inicial y que PHP no guardó nada.

Frases que podemos decir mientras demostramos

“Creamos un pedido ficticio después de validar sus campos en el navegador y en PHP.”

“La tabla se vuelve a dibujar desde el arreglo; no recargamos la página para actualizarla.”

“Al recargar se pierde la memoria de esta pestaña. Esa pérdida es un requisito de este laboratorio.”

Guion de exposición para los cuatro integrantes

Este reparto sirve para ensayar. No acredita quién escribió cada archivo. Al explicar aportes, cada integrante debe indicar las actividades que realmente realizó y la evidencia que pueda mostrar.

Carlos Andrés Chavarría Quirós

“Bicipedia representa una tienda de servicios y accesorios para bicicletas. Nuestro usuario puede preparar un pedido y consultarlo durante una misma sesión. Elegimos el dominio de tienda virtual permitido por el laboratorio. Tenemos cinco secciones y todos los precios y clientes son de demostración.”

Mostrar Inicio y Catálogo. Explicar los límites del proyecto y abrir la aplicación correcta desde el ZIP o la rama del laboratorio.

Carlos Josue Angúlo Xie

“HTML organiza los contenidos con etiquetas y formularios. CSS adapta las tarjetas y controles a móvil, tablet y computadora. Las etiquetas ayudan a entender cada campo. En móvil la tabla se desplaza dentro de una región, sin ampliar toda la página.”

Mostrar formulario, navegación con Tab y cambio de ancho. Crear los dos pedidos del recorrido. Saber localizar las secciones de `index.php` y las media queries.

Marco Vinicio Miranda Miranda

“Los pedidos están en `estado.solicitudes`. Buscar filtra lo que mostramos; editar reemplaza el elemento del mismo ID; eliminar deja fuera el ID elegido después de confirmar. Las funciones de renderizado actualizan la tabla y el dashboard.”

Buscar, editar y eliminar en pantalla. Abrir una función CRUD y explicar find, map o filter con un ejemplo.

Michael Aramis Cabrera Hodgson

“JavaScript revisa el formulario y manda un POST. PHP repite la validación y calcula con su catálogo. Devuelve JSON sin guardar pedidos. Las pruebas comprobaron el flujo y el reinicio; ahora recargamos para mostrarlo.”

Mostrar `fetch`, el archivo PHP y el cálculo. Recargar al final. Explicar qué se probó y qué debe comprobar el grupo en XAMPP.

Antes de cerrar, cada uno indica su aporte real. Si alguien solo estudió, probó o documentó una parte, debe describirlo así. Todos deben poder explicar la temporalidad, aunque el reparto de presentación sea diferente.

Preguntas posibles y respuestas para practicar

¿Dónde están los pedidos? En `estado.solicitudes`, un arreglo de JavaScript en la memoria de esta pestaña. No se guardan en PHP ni en una base de datos.

¿Por qué no se pierden al cambiar de sección? Porque mostramos otra sección del mismo documento mediante anclas y hidden; no lo recargamos.

¿Por qué desaparecen al recargar? Se vuelve a ejecutar el módulo y el arreglo inicia vacío. El catálogo reaparece porque está definido en el código.

¿Qué función crea y cuál dibuja? `crearSolicitud` modifica el arreglo; `renderizarSolicitudes` dibuja sus filas. Cambiar datos y mostrarlos son dos tareas distintas.

¿Cómo sabemos cuál editar? Cada pedido tiene un ID. `buscarSolicitud` lo encuentra y `editarSolicitud` cambia solo ese ID. El código se conserva.

¿PHP guarda el pedido? No. Recibe campos, valida, calcula y responde. JavaScript decide crear o actualizar en memoria cuando esa respuesta es correcta.

¿JSON cifra los datos? No. Es un formato de texto. `JSON.stringify` representa una lista como texto y `respuesta.json()` interpreta la respuesta.

¿Por qué hay dos catálogos? JavaScript necesita precios para mostrar el cálculo inmediato y PHP sus propios precios para validar el resumen. Cuando cambiamos un precio debemos mantener ambos iguales.

¿Qué pasa si PHP falla? No se agrega un pedido sin validar; aparece el error y el carrito queda para reintentar.

¿Qué significa el total del dashboard? La suma estimada de los pedidos presentes, de todos los estados. No son pagos ni ventas reales.

Un cambio breve que podemos ensayar

Si el docente pide cambiar el precio del kit, localizar `kit` en `js/datos-iniciales.js` y `php/catalogo.php`; modificar ambos al mismo valor. Recargar porque el catálogo se carga al abrir y crear un pedido para comprobarlo. No agregar almacenamiento para conservar los pedidos durante ese cambio. Este ejercicio se prepara para la defensa, no modifica la versión entregada por sí mismo.

Qué verificamos y qué debemos terminar

El 10 de octubre de 2026 se repitieron los 14 grupos de pruebas de la aplicación con PHP 8.5.11 y Edge mediante Playwright. Aprobaron creación, búsqueda, edición, eliminación, validación, cálculo, fallo HTTP, envío doble y reinicio. Las cinco secciones se comprobaron a 320, 375, 768, 1024 y 1280 px. Esta revisión no requirió cambiar las funciones de la aplicación.

Los diez casos que debemos poder repetir

Abrir en cero; crear válido; rechazar formulario vacío; buscar existente; mostrar búsqueda inexistente; editar; cancelar eliminación; confirmar eliminación; recalcular al cambiar cantidad; recargar y volver al estado inicial. La explicación anterior muestra cómo repetirlos sin instalar las herramientas automáticas.

Lista personal antes de exponer

- Abrimos la versión del laboratorio, no la rama main de Semana 1.
- Apache inicia y un pedido se valida desde localhost en el equipo de defensa.
- Cada integrante realiza por sí mismo el recorrido completo.
- Sabemos ubicar estado, una función CRUD, un render y el POST PHP.
- Explicamos los importes como valores ficticios de la propuesta.
- Podemos mostrar que cancelar conserva y que recargar elimina.
- Identificamos nuestros aportes reales sin atribuir trabajo que no hicimos.
- Conservamos el ZIP y verificamos el horario comunicado por el docente.

Documentos que delimitan el trabajo

Laboratorio 1, páginas 2 y 3: restricciones, datos temporales y tienda virtual. Páginas 5 a 7: etapas, funciones y PHP temporal. Páginas 8 y 9: estructura, ZIP, aceptación y pruebas. Páginas 10 y 11: rúbrica y defensa.

Tarea Semana 1: origen de la interfaz semántica, responsive y accesible. Guía TaskFlow: arreglos, eventos y DOM. Manual XAMPP y clase PHP: ejecución local y comunicación con servidor. Manual Motor de Pedidos: ejemplo de funciones y cálculos en Java; no es una obligación de integrar Java aquí. El mensaje de Laboratorio N.º 1 fija la defensa grupal y el horario.

Estos materiales ayudan a explicar el trabajo. La consigna concreta del laboratorio prevalece cuando un ejemplo de clase usa persistencia. No necesitamos agregar cifrado ni otros módulos para completar esta defensa.